Semi-Supervised Learning with Variational Auto-Encoder

Sanja Simonovikj
sanjas@mit.edu

Md Sanzeed Anwar
sanzeed@mit.edu

Adib Hasan
notadib@mit.edu

6.867 Final Project

Abstract

In this paper we explore variants of Variational Autoencoders (VAE) - we attempt to extend this popular generative framework to utilize labeled data. More precisely, we design and compare several such extended architectures by generating latent embeddings in a semi-supervised setting and testing their quality under a supervised framework. Our models, in particular the Gaussian Mixture Classifier VAE model, outperforms our baseline models, even with a small amount of labeled training data.

I Introduction

Variational Autoencoder is a powerful generative model that encodes inputs as probability distributions in the latent space. Samples from these distributions can be used later to generate new input data. Additionally, the embeddings in the latent space can capture structural information about the input that can be utilized to generate (predict) labels for the input. Kingma et al. developed semi-supervised Model 2 (M2) in [1] that uses some available label data to generate new labels in the semi-supervised settings.

However, one hurdle in both the standard VAE and the semi-supervised M2 is that the embeddings for various classes tend to be entangled together and display lower accuracy when used for classification. In our project, we demonstrate three extensions of the standard VAE that attempt to address this issue:

- *Gaussian Mixture Classifier VAE (A1):* VAE with a Gaussian Mixture prior and an added classifier component.
- *Standard Gaussian Classifier VAE (A2)* VAE with a standard Gaussian prior and an added classifier component.
- *Metric Learning VAE (A3):* VAE with a standard Gaussian prior and an added metric loss component in the loss function.

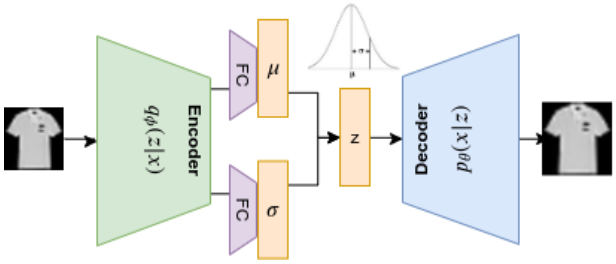

Figure 1: **Standard Variational Autoencoder; consists of Encoder $q_\phi(\mathbf{z}|\mathbf{x})$ and Decoder $p_\theta(\mathbf{x}|\mathbf{z})$. An input image is passed through the Encoder to obtain latent embedding which is passed to two separate fully connected networks to obtain latent mean vector μ and latent variance vector σ. These are used as parameters of a Gaussian distribution, from which we sample vector z which is passed through the Decoder to obtain a reconstructed image.**

We evaluate the quality of the embeddings generated by these architectures by evaluating the clustering quality in the latent space, the accuracy of a Random Forest Classifier trained on these embeddings, and the accuracy of the added classifier component if present in the architecture. For comparison purposes, we also present the performance of our baseline models under these metrics.

II Related work

A Variational Autoencoders

The framework of *Variational Autoencoders* (Fig. 1), first introduced by Kingma and Welling [2], provides a method for jointly learning deep latent-variable models and corresponding inference models using stochastic gradient descent. More precisely, with VAEs we can learn stochastic mapping between observed $\mathbf{x}$-space of dataset $\mathcal{D}$, with complicated empirical distribution $q_{\mathcal{D}}(\mathbf{x})$, and latent $\mathbf{z}$-space, whose

distribution is simple (often chosen to be isotropic Gaussian). The *generative* model learns a joint distribution $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{z})p_\theta(\mathbf{x}|\mathbf{z})$, factorized into a prior distribution over latent space, $p_\theta(z)$, and a stochastic **decoder**, $p_\theta(\mathbf{x}|\mathbf{z})$, parametrized using a deep neural network. The stochastic **encoder** $q_\phi(\mathbf{z}|\mathbf{x})$, also known as *inference* model and also parametrized with a deep neural network, approximates the true but intractable posterior $p_\theta(\mathbf{z}|\mathbf{x})$ of the generative model.

The optimization objective of the VAE, like in other variational methods, is the *evidence lower bound* $\mathcal{L}_{\theta,\phi}(\mathbf{x})$, or ELBO, given by:

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) = \log p_\theta(\mathbf{x}) - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z}|\mathbf{x})) \quad (1)$$

which can be conveniently written as

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) = \mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z})) \quad (2)$$

Note that since the KL-divergence is always non-negative, we can conclude from eq. 1 that the ELBO is a lower bound on the likelihood $\log p_\theta(\mathbf{x})$ of the data. Maximization of the ELBO w.r.t. the parameters θ and ϕ will concurrently maximize the likelihood of the data and minimize the KL-divergence between the approximation $q_\phi(\mathbf{z}|\mathbf{x})$ and the true posterior $p_\theta(\mathbf{z}|\mathbf{x})$. In practice the optimization of 2 is made efficient by using stochastic gradient descent methods and applying the so-called *reparametrization trick*, which makes the differentiation w.r.t. ϕ, which affects the objective $\mathcal{L}_{\theta,\phi}$ through the random variable $\mathbf{z}$, possible. For more detailed derivation of the above equations we ask the reader to refer to the paper [2].

B Semi-supervised learning

With the increasing amount of data in today's world and the expense of providing label information, the question becomes how can one make use of the abundance of data that is unlabeled? One of the ways that semi-supervised learning methods attempt to solve this problem is by incorporating label information into deep generative models such as VAEs which are normally fully unsupervised and can model the underlying data distribution. Kingma et al. [1] propose deep generative models that allow for effective generalisation from small labeled data sets to large unlabeled ones. Our work is tightly related to this referenced work, yet it varies in the exact architecture of the deep models and the dataset used. The details about this will be explained in subsequent sections.

III Models

In this section we describe the key principles behind the baseline model, as well as the three successful architectures we propose. For this discussion, we assume that we are given data $\mathcal{D} = (X, Y)$, where x_i and y_i respectively represent the i-th datapoint and its corresponding label. We also assume that $\forall y_i \in Y, y_i \in \{1, 2, \cdots, k\}$, i.e. there are k possible classes. The data $\mathcal{D}$ is divided into two sets, namely, $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$. $\mathcal{D}_{\text{train}}$ is further divided into two subsets: $\mathcal{D}_{\text{labeled}}$ and $\mathcal{D}_{\text{unlabeled}}$. It is assumed that the labels y_i from $\mathcal{D}_{\text{unlabeled}}$ are unavailable during training, and labels from $\mathcal{D}_{\text{test}}$ are unavailable during both training and testing, and can be used only for computing model accuracy.

A Baseline model

Our baseline model is a standard Variational Autoencoder with convolutional and deconvolutional layers in the Encoder and the Decoder respectively. The Encoder turns the 28x28 input image into a latent vector by first creating a mean vector and a standard deviation vector, which are used as parameters to a Gaussian distribution from which we sample the final 20-dimensional latent vector.

The Encoder consists of two convolutional layers with kernel size 4, padding 15 and stride 2. We use two fully connected layers with hidden dimension 1024 to obtain the latent mean vector. Different set of two fully connected layers is used to obtain the latent standard deviation vector. Note that the convolutional layers used to obtain these latent vectors are shared. The activation function that we used for all relevant layers is ReLU.

The Decoder takes the reparametrized latent vector which goes through two fully connected layers with hidden dimension 1024, followed by two deconvolutional layers with kernel size 4, padding 1 and stride 2. A ReLU activation is used on all relevant layers except the output layer, where we use a Sigmoid activation function, which naturally squeezes the output pixels to be between 0 and 1.

Note that we chose this as our baseline, as opposed to vanilla CNN for example, because the goal of this project is producing better embeddings by utilizing labeled data, as opposed to performing better classification with less data.

B Gaussian Mixture Classifier VAE Architecture (A1):

We now propose a probabilistic semi-supervised model and its objective. First, we derive the objec-

tive function and then describe a possible implementation. Our probabilistic model will be as follows:

$$p(\mathbf{y}) = \mathrm{Cat}(\mathbf{y}|\pi); \quad p(\mathbf{z}|\mathbf{y}) = \mathcal{N}(\mu_{\mathbf{y}}, \mathbf{I})$$

where $\mathrm{Cat}(\mathbf{y}|\pi)$ denotes the categorical distribution of labels with parameter π. We also assume that both $p_\theta(\mathbf{x}|\mathbf{z})$ and $p_\theta(\mathbf{y}|\mathbf{z})$ are suitable, non-linear function of their parameters (eg. Gaussian, Bernoulli etc). Also notice that $p(\mathbf{z})$ is a Gaussian Mixture:

$$p(\mathbf{z}) = \sum_{\mathbf{y}} p(\mathbf{y})\mathcal{N}(\mu_{\mathbf{y}}, \mathbf{I})$$

To construct the approximate posterior distribution, we consider the following models:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu_\phi(\mathbf{x}), (\sigma_\phi(\mathbf{x}))^2\mathbf{I})$$

$$q_\phi(\mathbf{y}|\mathbf{z}) = \mathrm{Cat}(\mathbf{y}|\pi_{\mathbf{z}})$$

$$q_\phi(\mathbf{x}, \mathbf{y}|\mathbf{z}) = q_\phi(\mathbf{x}|\mathbf{z})q_\phi(\mathbf{y}|\mathbf{z})$$

We know that the variational loss can be written as:

$$-L_{VAE} = -KL(q_\phi(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z})) + \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[p_\theta(\mathbf{x}|\mathbf{z})]$$

Thus for a single unlabeled datapoint, the variational objective is simply $\log p(\mathbf{x}) \geq -L_{VAE}$. Similarly for a single labeled datapoint

$$\begin{aligned}\log p(\mathbf{x}, \mathbf{y}) &= \log p_\theta(\mathbf{y}|\mathbf{x}) + \log p(\mathbf{x}) \\ &\geq \log p_\theta(\mathbf{y}|\mathbf{x}) - L_{VAE}\end{aligned}$$

$\log p(\mathbf{y}|\mathbf{x})$ can be interpreted as the log likelihood of a classifier that predicts labels. This can be approximated by any reasonable distribution of $\mathbf{x}$ and $\mathbf{y}$. However, to ensure that $p(\mathbf{y}|\mathbf{x})$ depends on latent variable, we approximate it with $q_\phi(\mathbf{y}|\mathbf{x}, \mathbf{z})$. Now, since $q_\phi(\mathbf{x}, \mathbf{y}|\mathbf{z}) = q_\phi(\mathbf{x}|\mathbf{z})q_\phi(\mathbf{y}|\mathbf{z})$, it can be seen that

$$q_\phi(\mathbf{y}|\mathbf{x}, \mathbf{z}) = \frac{q_\phi(\mathbf{x}, \mathbf{y}|\mathbf{z})}{q_\phi(\mathbf{x}|\mathbf{z})} = q_\phi(\mathbf{y}|\mathbf{z})$$

Hence, we can use $\log q_\phi(\mathbf{y}|\mathbf{z})$ as an approximation of this classifier component. Therefore, the overall objective for training dataset becomes

$$-L_{A1} = \sum_{\mathcal{D}_{\text{train}}} -L_{VAE}(\mathbf{x}, \mathbf{z}) + \lambda \sum_{\mathcal{D}_{\text{labeled}}} \log q_\phi(\mathbf{y}|\mathbf{z}) \tag{3}$$

where λ is a hyperparameter that scales the losses during training.

Implementation. We start with the VAE architecture from baseline model, and change change the prior $p(\mathbf{z})$ of a standard VAE to be a Gaussian Mixture (the KL divergence closed form changes, which we shall derive below). Then we attach a classifier to

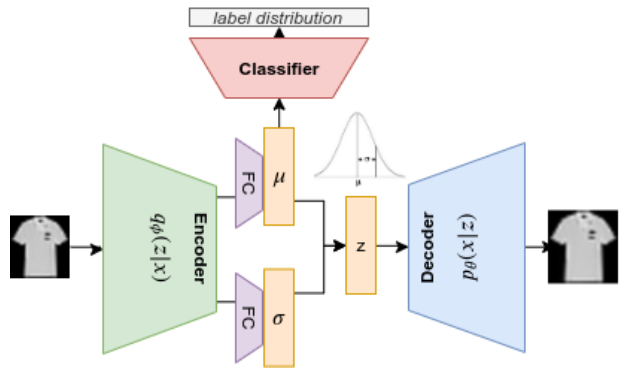

Figure 2: **Architecture of the model A2 – Standard Gaussian Classifier VAE model. Model A1 – Mixture Gaussian Classifier VAE model, uses same architecture but has different priors. An input image is passed through the Encoder and the resulting latent mean vector μ is used as a feature vector for a classifier. The sampled vector z is passed through the Decoder as usual to obtain a reconstructed image. The training is done by jointly optimizing all three components of the architecture: Encoder, Decoder and Classifier.**

it that takes $z = \mu_\phi(\mathbf{x})$ as input, as shown in figure 2. The classifier has two fully connected layers, one taking output to 100 dimension with ReLU activation and the other reducing it down to 10 dimension with Softmax activation. The loss function for classifier is standard NLL (Negative-Log Likelihood) loss, which is substituted into Equation 3 for computing loss of the entire model. We have used hyperparameter $\lambda = 10$ in Equation 3 and the standard reparameterization trick used in VAE. (Kingma et al. [2]) throughout the training.

Computing KL Divergence. At first we shall define our priors. To simplify computation, we assume the following

$$p(\mathbf{y} = i) = \frac{1}{k} \quad \text{for } 1 \leq i \leq k$$

So, $p(z) = \frac{1}{k}\sum_{i=1}^{k}\mathcal{N}(\mu_i, \mathbf{I})$. Let J be the dimensionality of $\mathbf{z}$. Also, we shall write $\mu_\phi(\mathbf{x})$ and $\sigma_\phi^2(\mathbf{x})\mathbf{I}$ as $\mu_{\mathbf{x}}$ and $\sigma_{\mathbf{x}}^2\mathbf{I}$, respectively, for brevity. Now,

$$\begin{aligned}&\int q(\mathbf{z}|\mathbf{x}) \log p(\mathbf{z}) d\mathbf{z} \\ &= \int \mathcal{N}(\mu_{\mathbf{x}}, \sigma_{\mathbf{x}}^2\mathbf{I}; \mathbf{z}) \frac{1}{k} \sum_{i=1}^{k} \log \mathcal{N}(\mu_i, \mathbf{I}; \mathbf{z}) d\mathbf{z} \\ &= \int \mathcal{N}(\mu_{\mathbf{x}}, \sigma_{\mathbf{x}}^2\mathbf{I}; \mathbf{z}) \Big(-\frac{J}{2} \log(2\pi)\end{aligned}$$

$$
\begin{aligned}
&\qquad\qquad - \sum_{i=1}^{k} \frac{||\mu_i||^2 - 2\mu_i \mathbf{z} + ||\mathbf{z}||^2}{2k}\Big) \\
&= -\frac{J}{2}\log(2\pi) - \frac{||\mu_{\mathbf{x}}||^2 + \mathrm{Tr}(\sigma_{\mathbf{x}}^2 \mathbf{I})}{2} \\
&\qquad\qquad - \sum_{i=1}^{k} \frac{||\mu_i||^2 - 2\mu_i^T \mu_{\mathbf{x}}}{2k}
\end{aligned}
$$

Similarly,

$$
\begin{aligned}
&\int q(\mathbf{z}|\mathbf{x}) \log q(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\
&= -\frac{J}{2}\log(2\pi) - \frac{\log\det(\sigma_{\mathbf{x}}^2 \mathbf{I})}{2} \\
&\quad - \frac{1}{2}\int \mathcal{N}(\mu_{\mathbf{x}}, \sigma_{\mathbf{x}}^2 \mathbf{I}; \mathbf{z})(\mathbf{z} - \mu_{\mathbf{x}})^T (\sigma_{\mathbf{x}}^2 \mathbf{I})^{-1} (\mathbf{z} - \mu_{\mathbf{x}}) d\mathbf{z} \\
&= -\frac{J}{2}\log(2\pi) - \frac{\log\det(\sigma_{\mathbf{x}}^2 \mathbf{I})}{2} \\
&\qquad\qquad - \frac{1}{2}\mathbb{E}[\mathrm{Tr}((\sigma_{\mathbf{x}}^2 \mathbf{I})^{-1} ||\mathbf{z} - \mu_{\mathbf{x}}||^2)] \\
&= -\frac{J}{2}\log(2\pi) - \frac{\log\det(\sigma_{\mathbf{x}}^2 \mathbf{I})}{2} \\
&\qquad\qquad - \frac{1}{2}\mathrm{Tr}((\sigma_{\mathbf{x}}^2 \mathbf{I})^{-1} \mathbb{E}[||\mathbf{z} - \mu_{\mathbf{x}}||^2]) \\
&= -\frac{J}{2}\log(2\pi) - \frac{\log\det(\sigma_{\mathbf{x}}^2 \mathbf{I})}{2} - \frac{1}{2}\mathrm{Tr}((\sigma_{\mathbf{x}}^2 \mathbf{I})^{-1} (\sigma_{\mathbf{x}}^2 \mathbf{I})) \\
&= -\frac{J}{2}\log(2\pi) - \frac{\log\det(\sigma_{\mathbf{x}}^2 \mathbf{I})}{2} - \frac{J}{2}
\end{aligned}
$$

Therefore, $-KL(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$ would be

$$
\begin{aligned}
&\int q(\mathbf{z}|\mathbf{x}) \log p(\mathbf{z}) d\mathbf{z} - \int q(\mathbf{z}|\mathbf{x}) \log q(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\
&= \frac{1}{2}\left(\log\det(\sigma_{\mathbf{x}}^2 \mathbf{I}) + J - ||\mu_{\mathbf{x}}||^2 - \mathrm{Tr}(\sigma_{\mathbf{x}}^2 \mathbf{I})\right) \\
&\quad - \sum_{i=1}^{k} \frac{||\mu_i||^2 - 2\mu_i^T \mu_{\mathbf{x}}}{2k}
\end{aligned}
$$

In the implementation below, $J = 20$, $k = 10$, $\mu_{\mathbf{x}}$ is generated by the encoder and $\mu_i = \underbrace{0, \ldots, 0}_{2(i-1)} 1, 1, 0, \ldots, 0$ for $1 \leq i \leq k$.

The A1 model is different from the M2 model introduced by Kingma et al. in [1]. The prior in M2 model is $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$, but we consider this prior to be a Gaussian Mixture dependent on $\mathbf{y}$. Furthermore, M2 takes ELBO bound on $p(\mathbf{x}, \mathbf{y})$. Consequently. its variational loss is a function of $\mathbf{y}$ and has multiple terms. In contrast, our loss function is much simpler and the VAE loss of our model does not depend on $\mathbf{y}$.

C Standard Gaussian Classifier VAE Architecture (A2)

We apply the same idea from A1, but this time with a standard Gaussian prior on $\mathbf{z}$. This model again adds a Classifier component on top of the standard VAE architecture. More precisely, we take the latent mean vector $\mu_\phi(\mathbf{x})$ of the latent random variable $\mathbf{z}$ and use that as a feature vector in a fully connected network with 100 hidden units to obtain the log of a probability distribution over the labels (using Log-Softmax output layer). The architecture is illustrated in Figure 2.

The training of this model is done by jointly optimizing the standard VAE and the Classifier component with the following loss function:

$$
L_{A2} = L_{VAE} + \lambda L_{Classifier} \tag{4}
$$

where L_{VAE} is the standard VAE loss and $L_{Classifier}$ is the Negative Log-Likelihood (a.k.a. Cross Entropy Loss). The parameter λ controls how much weight is given to the classifier and can serve to bring the two losses on the same scale. We found the value $\lambda = 10$ to work best in our experiments.

The motivation behind this model stems from the assumption that a Classifier component will help in pushing the embeddings of different classes to be distant from each other, i.e. be more disentangled in latent space.

D Metric Learning VAE Architecture (A3)

Metric Learning VAE model uses the standard VAE architecture, but adds a metric learning component in the loss function. More precisely, the training of this model is done by optimizing the standard VAE with the following loss function:

$$
L_{A3} = \rho L_{VAE} + L_{Metric} \tag{5}
$$

where L_{metric} is given by

$$
\begin{aligned}
L_{metric} = \sum_{\substack{i,j:x_i,x_j \\ \text{labeled}}} \Big[&(1 - [[y_i = y_j]]) \cdot \frac{1}{1 + ||x_i - x_j||^2} \\
&+ [[y_i = y_j]] \cdot ||x_i - x_j||^2\Big]
\end{aligned} \tag{6}
$$

The parameter ρ controls how much weight is given to the VAE loss component, and serves to scale the two components of the loss function. We found $\rho = 10000$ to be optimal in our experiments.

The intuition behind this model is that (i) if two images with the same label are mapped too far away

in the latent space, their contribution in the metric loss will be high, and (ii) if two images with different labels are mapped too close in the latent space, their contribution in metric loss will be high. In other words, this extra component in the loss can help us achieve better spacial separation in the latent space.

IV Setup

A Dataset

In all of our experiments we use the Fashion-MNIST dataset [3] comprising of 28x28 grayscale images of 70,000 fashion products from 10 categories, with 7,000 images per category (see Fig. 3). The training set has 60,000 images and the test set has 10,000 images. As a pre-processing step, we scale the pixels of the images to be between 0 and 1. We chose this dataset as it is well curated yet more challenging than the standard MNIST (digits) dataset [4].

B Semi-Supervised train split

We split the training dataset into two datasets, one with labels and the other without labels. For each of the models we tested, the labels from the labeled dataset are visible to the model during training, but the labels from unlabeled dataset are not. A model that does not use dataset labels during training (eg. Vanilla VAE) may ignore labels from both labeled dataset and unlabeled dataset. The exact procedure for splitting the training dataset is the following:

First, we set a percentage $0 < p < 100$ such that the size of the labeled dataset is approximately $p\%$ of the training dataset. Then the training dataset is shuffled once. After that, starting from the beginning of the training dataset, first $60,000 \times p/10$ data points from each class is added to the labeled dataset and the rest is added to unlabeled dataset. Then, we iterate over both labeled and unlabeled datasets to ensure class balance.

We also seed the random shuffling step to ensure that same dataset is used across the models. Also, we experiment with the following values for p: 0.5, 1, 5, 10, 20, 40, 50, 75, 99.

C Hyperparameters

Throughout our experiments we aim to maintain the same hyperparameters, so that our results are comparable. We train our models for 10 epochs with Adam optimizer and learning rate 10^{-4} with batch size of 128. The dimension of our latent vector is set to 20. Any deviation from these hyperparameters will be explicitly noted.

Label	Description	Examples
0	T-Shirt/Top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandals	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boots	

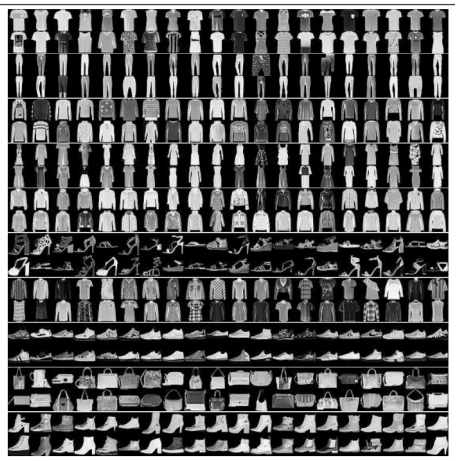

Figure 3: **Class names and example images in Fashion-MNIST dataset (Image credit: Xiao et al. [3])**

D Evaluation method

We use the mean vectors μ of the latent variables $\mathbf{z}$ obtained by the VAE encoder as features for our image data in all of our evaluation methods.
We evaluate and compare our models according to the following criteria:

1. **Random forest (RF) classification accuracy** [5]. We use the features of the train data to train a Random Forest classifier and report the accuracy on the test data. Note that for training the Random Forest we use all labels of the train data. We choose Random Forests as one way to evaluate our latent embeddings as they are not only fast and easy to implement but they can handle high dimensional spaces as well as large number of training examples.

2. **Davies-Bouldin score** [6]. The score is defined as the average similarity measure of each cluster with its most similar cluster, where similarity is the ratio of within-cluster distances to between-cluster distances. Thus, clusters which are farther apart and less dispersed will result in a better score. The minimum score is zero, with lower values indicating better clustering. We report the clustering score on the test data.

3. **Accuracy of the predicted labels:** For the models that support generating (predicting) la-

labeled	VAE	M2	A1	A2	A3
0.5%	83.56	67.32	83.57	78.5	82.78
1%	83.56	64.70	84.29	80.38	83.36
5%	83.56	63.53	84.16	82.41	83.04
10%	83.56	53.99	85.05	84.40	82.65
20%	83.56	54.74	86.63	85.68	84.15
40%	83.56	56.55	86.93	85.86	85.41
50%	83.56	57.19	88.34	88.44	86.78
75%	83.56	62.12	89.35	89.43	87.17
99%	83.56	58.13	89.18	89.84	87.61

Table 1: **Random Forest accuracy of the embeddings for various models and different amounts of labeled data**

bels, for instance M2, A1, A2, we train the models with various percentage of labeled training data. We calculate the accuracy of the predicted labels for the test dataset. The Standard VAE model cannot predict labels. Because we still want to have a baseline in this case, we train a Random Forest Classifier using the embeddings from Standard VAE and with various percentages of training labels. This forms our baseline VAE+RF for this metric.

We note that we run our models for 10 epochs but we report the three evaluation values for the best performing epoch with respect to the first evaluation criteria, that is the Random Forest classification accuracy.

V Results

The performance of the five models mentioned in our discussions measured under the evaluation metrics described in section IV(D) are demonstrated in tables 1, 2 and 3, respectively. In addition, Figure 4 demonstrates a visual comparison of the RF classification accuracy scores for all five models.

As seen from the numerical data, the standard VAE model has an RF classification accuracy of 83.56% for any percentage of labeled data, since it does not utilize the labels.

The A1 model achieves a similar RF classification accuracy as as our baseline model (the standard VAE) with very small percentage of labeled data (e.g. 0.5%). The accuracy increases as we increase the percentage of labeled data, and in particular, we see the accuracy increases by ∼5% if we use 50% labeled data.

The A2 model performs better than the A1 model if we use high percentages of labeled data, but performs

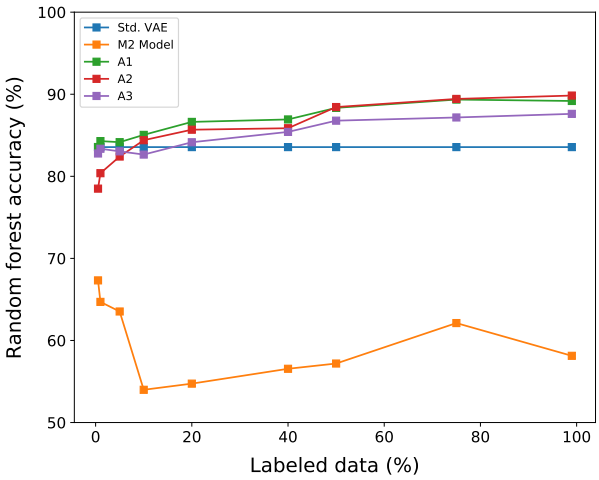

Figure 4: **Classification accuracy of a random forest classifier trained on embeddings generated from various architectures, plotted against the corresponding percentages of labeled data used in generating those embeddings. Each curve represents a different architecture. Even with a small percentage of labeled data, the models we propose tend to perform significantly well.**

worse than the A1 model if the percentage of labeled data is too low. This demonstrates that a Gaussian Mixture prior is very good at indicating the classifier component to promote disentanglement for small amount of training data. However, as the percentage of training data increases, the classifier depends less on the prior.

The A3 model displays a similar pattern in its performance as we increase the percentage of labeled data. While it achieves worse RF classification accuracy than the A1 and the A2 models, it performs better in the Davis-Bouldin score metric compared to A1 or A2.

The M2 model seems to perform poorly in our cluster quality metric, which comes at no surprise because the embeddings generated by M2 are very entangled as shown in Figure 7.

Next, we compare the accuracy of the generated (predicted) labels from different models. The standard VAE cannot generate labels. In order to still have a baseline in this case, we train a Random Forest classifier on its embeddings with only p% of the labels and we evaluate it on test data. Note that this scenario is different from the result for VAE in Table 1, where the Random Forest Classifier was trained on all of the labels (as done with the RF for all other models). We can see that the A1 model performs the

labeled	VAE	M2	A1	A2	A3
0.5%	3.35	2.39	2.39	2.57	2.781
1%	3.35	2.37	1.92	2.39	2.504
5%	3.35	2.14	2.01	2.21	2.163
10%	3.35	2.38	1.85	2.59	2.181
20%	3.35	2.34	1.43	1.7	1.606
40%	3.35	2.45	1.77	2.15	1.855
50%	3.35	2.54	1.41	1.45	0.917
75%	3.35	2.67	1.43	1.34	0.869
99%	3.35	2.45	1.39	1.41	0.825

Table 2: **Davies-Bouldin score for various models and different amounts of labeled data**

best among the 4 models with very small percentage (0.5%-1%) labeled data, but the A2 model catches up very quickly with high amount of labeled data.

labeled	VAE+RF	M2	A1	A2
0.5%	62.53	38.55	73.04	60.28
1%	67.91	43.92	78.00	71.84
5%	76.73	71.61	76.71	61.59
10%	78.31	80.32	80.97	71.15
20%	80.80	81.47	85.65	81.11
40%	81.94	82.33	85.79	71.26
50%	82.05	83.47	88.84	87.97
75%	82.71	83.95	89.35	89.04
99%	83.56	85.81	89.90	89.79

Table 3: **Accuracy(%) of predicted test set labels for different amounts of labeled data**

VI Other Explored Models

We have tried several other models throughout the course of this project, which failed to produce viable results. One of the approaches was to replace the prior for z to a Gaussian mixture of 10 Gaussians with different (predefined) means and identity covariance matrix on standard VAE. This changes the KL divergence term of the loss function. The motivation behind this approach was to encourage the embeddings to be implicitly clustered by their labels. However, it seems from the experiment that the encoder learned to distribute the latent variable only around one of the Gaussians from the Gaussian mixture. Hence, both the classifier component and Gaussian Mixture prior are necessary for our A1 model.

Other model we attempted was to incorporate label information into Classifier VAE architecture by concatenating 10 extra one-hot channels in the input and use that instead of the single channel for the grayscale

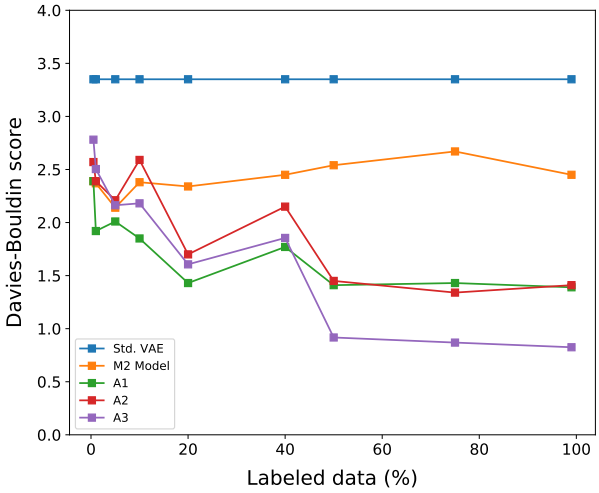

Figure 5: **Davies-Bouldin score of the latent embeddings generated from various architectures, plotted against the corresponding percentages of labeled data used in generating those embeddings. Each curve represents a different architecture. Even with a small percentage of labeled data, the models we propose tend to achieve better separation. As we increase the amount of labeled data, the separation gradually becomes better.**

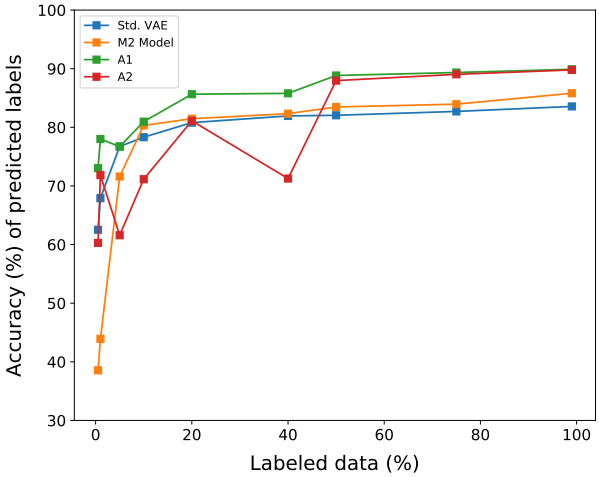

Figure 6: **Accuracy of predicted test set labels for various architectures, plotted against the corresponding percentages of labeled data used in generating those embeddings. Each curve represents a different architecture. Even with a small percentage of labeled data, the classifier component in our proposed models performs significantly better.**

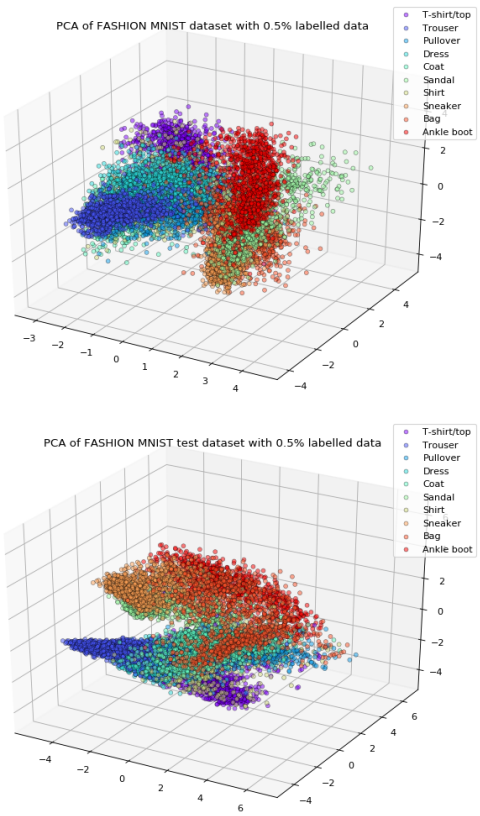

Figure 7: **PCA plot of the latent embeddings of test data generated by A1 (top) and M2 (bottom) from 0.5% labeled data. Embeddings generated by A1 show tighter clustering by the labels.**

image. During testing phase we used predicted label information from a pretrained classifier. While this approach did produce very disentangled clusters in latent space, it was overly dependent on the labels, which is not surprising given that the majority of the input ($\frac{10}{11}$) is label information. Thus, any misclassification from the pretrained classifier resulted in misplacement into the wrong cluster, which was reflected in the Random Forest which had identical accuracy as the pretrained classifier alone. Further research could potentially find better ways to make the model less dependent on the label information during test phase while still benefiting from label information during train phase in a manner that preserves the disentanglement.

VII Discussion & Conclusion

In this project we have proposed several architectures which extend the vanilla VAE architecture to include label information, while still making use of all the unlabeled data to produce embeddings which can be used for downstream tasks such as classification. We found all of them to perform nontrivially better than our baseline even with small amounts of data. Gaussian Mixture Classifier VAE can predict labels with high accuracy from very few (0.5-1%) labeled training datapoints and generally outperforms the standard VAE and M2 model from [1] on Fashion MNIST dataset.

Obtaining disentangled embeddings of images by having access to only a small amount of labeled data while still utilizing the unlabeled one opens the door for applications beyond classification, including but not limited to adversarial robustness, representation learning and compression. As such it is a very interesting and useful problem that should be investigated further by the research community.

VIII Acknowledgements

We would like to thank Prof. David Sontag and our TA Matthew McDermott for their valuable suggestions throughout the realization of this project. We would also like to thank the whole 6.867 course staff for their continued support throughout the semester.

References

[1] Diederik P. Kingma, Danilo Jimenez Rezende, Shakir Mohamed, and Max Welling. Semi-supervised learning with deep generative models. *CoRR*, abs/1406.5298, 2014.

[2] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. *CoRR*, abs/1312.6114, 2013.

[3] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *CoRR*, abs/1708.07747, 2017.

[4] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, Nov 1998.

[5] Leo Breiman. Random forests. *Mach. Learn.*, 45(1):5–32, October 2001.

[6] Wikipedia contributors. Davies–bouldin index — Wikipedia, the free encyclopedia, 2019. [Online; accessed 26-November-2019].